An Introduction to Universal Artificial Intelligence

Chapter 13: Computational Aspects

Based on Hutter, Quarel & Catt

CRC Press, 2024

"Computing is not about computers any more. It is about living."
— Nicholas Negroponte

Outline

Why Ask About Computability, and What We Already Know I

The two previous chapters covered *efficiently computable* approximations of AIXI: MDP-AIXI (Chapter 11) and MC-AIXI-CTW (Chapter 12). Both replace the incomputable universal mixture ξ (xi) by a computable one over a restricted class of environments, and replace the incomputable *expectimax* search by a feasible algorithm (dynamic programming or Monte Carlo Tree Search).

Clearly AIXI itself is incomputable — but *where exactly* does it sit in the computability hierarchy? Recall from Section 2.6.3 the **arithmetic hierarchy** Δ_n^0 (Delta): a set or function is Δ_1^0 iff it is ordinarily computable; it is Δ_2^0 (*approximable*) iff a computable process can produce a sequence of guesses that eventually converges to the right answer (without ever telling us when it has converged); $\Delta_3^0, \Delta_4^0, \ldots$ are successively weaker notions, each allowing one more layer of "in the limit of a limit."

Goal of this chapter. In Section 13.1 we pin down the *exact* level of (in)computability of AIXI and several of its variants:

- **Upper bounds.** Optimal value functions, optimal policies, and ε-optimal (epsilon, "almost optimal") policies of AIXI and its variants all lie in Δ_n^0 for $n = 2$ or $n = 3$.

Why Ask About Computability, and What We Already Know II

- **Lower bounds.** None of these agents — not even the weakest variant — is computable; hence none of them is in Δ_1^0.

Section 13.2 then introduces **AIXI**tl, a theoretically optimal *computable* approximation of AIXI that runs in bounded time t and uses programs of bounded length l — the price we pay is that its runtime is astronomically, but finitely, large.

Setting the Stage: Semimeasures, Discount, and $\mathcal{M}_{sol}$ I

Before discussing the computability of *optimal policies*, we first need the computability of *value functions* (recall Section 2.6 for the underlying notions of computability). We focus on the class of **lower semicomputable environments** $\mathcal{M}_{sol}$ ("sol" for Solomonoff): all semimeasures that can be approximated from below by an algorithm.

Setting the Stage: Semimeasures, Discount, and $\mathcal{M}_{sol}$ II

Recap of the objects involved

- ν (nu): a *semimeasure* — an environment model that assigns a (possibly sub-normalized) probability $\nu(e_{<t} \| a_{<t})$ to a percept history given the actions taken; "semi" because the total probability mass may be less than 1 (some probability can silently "leak away").
- γ_k (gamma sub k): the discount weight put on the reward received at time k; Γ_k (Gamma sub k), the **discount normalizer**, is defined so that $\Gamma_k := \sum_{i \geq k} \gamma_i$, which keeps values bounded in $[0, 1]$ regardless of how many future rewards are summed.
- $\mathcal{R} \subset [0, 1]$: the finite set of possible reward values.
- $V^*_\nu(h_{<t})$, $Q^*_\nu(h_{<t}, a_t)$: the optimal (recursively defined, Theorem 6.7.2) value and Q-value functions — the best expected future discounted reward achievable from history $h_{<t}$ (respectively, after also committing to action a_t).

Solomonoff's universal mixture $\xi_U = M$ (Definition 3.7.1) is the most important member of $\mathcal{M}_{sol}$: it is the Bayes-mixture AIXI uses as its "best guess" over all lower semicomputable environments.

Theorem 13.1.1: Computability of V_ν^* and Q_ν^*

Theorem 13.1.1 (Computability of V_ν^* and Q_ν^*)

Let semimeasure ν, the discount normalizer Γ_k, and the finite reward set $\mathcal{R} \subset [0,1]$, all be Δ_n^0-computable. Then the (recursive) optimal (Q-)value functions $V_\nu^*(h_{<t})$ and $Q_\nu^*(h_{<t}, a_t)$ are Δ_n^0-computable.

Plain English: if all the "ingredients" that go into defining the value function (ν, the discounting schedule, and the possible rewards) can be computed to level n in the arithmetic hierarchy, then the value function built out of them (via the Bellman-style recursion of Theorem 6.7.2) does not get any harder to compute — it stays at exactly the same level n.

Why this matters. V_ν^* and Q_ν^* are the quantities an optimal agent needs in order to decide what to do. This theorem says: computing "how good is the best possible future from here" is no harder than computing the environment itself.

Remark 13.1.2 and Proof Sketch of Theorem 13.1.1 I

Remark 13.1.2 (Original V, Q for semimeasures can be more complex)

If we assume ν to be a (full) measure, then there is no difference between the recursive formulation (Theorem 6.7.2) and the original Definition 6.6.1 and Definition 6.7.1. But if ν is a genuine *semi*measure, the definitions differ, and we explicitly have to take the limit $m \to \infty$ in the original definition, which pushes $V^*_\nu(h_{<t})$ up to Δ^0_{n+1}. For finite- or moving-horizon agents (Definition 6.4.2) we always have $V \in \Delta^0_n$, since no $m \to \infty$ limit needs to be taken.

Taking a limit as $m \to \infty$ is exactly the operation that costs you one extra level of the arithmetic hierarchy (recall Section 2.6.4): $\lim \Delta^0_n = \Delta^0_{n+1}$.

Remark 13.1.2 and Proof Sketch of Theorem 13.1.1 II

Proof sketch of Theorem 13.1.1.

(i) From Theorem 6.7.2, $V_\nu^{*,m}(h_{<t})$ (the value function truncated to a horizon of m steps) is a *finite-depth* $(m-t)$-step recursion built from continuous combinations (sum, max, product, ratio) of finitely many terms, and each term is assumed Δ_n^0. Hence $V_\nu^{*,m}(h_{<t}) \in \Delta_n^0$ by Theorem 2.6.19 (combining finitely many Δ_n^0 functions with computable operations keeps you in Δ_n^0).

(ii) Recall a real-valued function is *computable* (*estimable*) iff there is an algorithm that, given the input and any $\varepsilon > 0$, computes the function to within ε in finite time. Assume $\nu, \Gamma, \mathcal{R} \in \Delta_1^0$ (estimable). From Lemma 6.7.6,

$$0 \;\le\; V_\nu^*(h_{<t}) - V_\nu^{*,m}(h_{<t}) \;\le\; \Gamma_{m+1}/\Gamma_t.$$

Since $\Gamma_m \to 0$ as $m \to \infty$ (Definition 6.4.1), for $m = t, t+1, \ldots$ we estimate Γ_{m+1}/Γ_t to accuracy $\varepsilon/4$ until we find an m for which this estimate is $< \varepsilon/2$; for that m we estimate $V_\nu^{*,m}(h_{<t})$ to accuracy $\varepsilon/4$. Altogether we have estimated $V_\nu^*(h_{<t})$ to accuracy ε.

Remark 13.1.2 and Proof Sketch of Theorem 13.1.1 III

(iii) If $f(z_1, z_2, \dots)$ is computable (estimable) and $g_i(x_i)$ are Δ_n^0-computable, then $f(g_1(x_1), g_2(x_2), \dots)$ is Δ_n^0-computable. This lifts part (ii): if $\nu, \Gamma, \mathcal{R} \in \Delta_n^0$, then $V_\nu^*(h_{<t}) \in \Delta_n^0$.

(iv) The same argument holds for Q_ν^*, which is just V_ν^* with the left-most $\max_{a_t}$ removed. ■

The key trick in (ii): the proven error bound Γ_{m+1}/Γ_t tells us exactly how deep (m) we must unroll the recursion to guarantee any desired accuracy ε — this is what makes V_ν^* "estimable" rather than merely "defined as a limit."

Theorem 13.1.3: Lower Semicomputability of V^*_ν, Q^*_ν I

Theorem 13.1.3 (Lower semicomputability of V^*_ν and Q^*_ν)

(i) If $\nu(e_k|\cdots), \gamma_k, \mathcal{R} \in \Sigma^0_n$, then recursive $\Gamma_t V^*_\nu(h_{<t}) \in \Sigma^0_n$.

(ii) If $\nu(\cdots), \gamma_k, \mathcal{R} \in \Sigma^0_n$, then recursive $\nu(e_{<t}\|a_{<t})\Gamma_t V^*_\nu(h_{<t}) \in \Sigma^0_n$.

Σ^0_n (Sigma) is the "can be confirmed from below" half of the arithmetic hierarchy: an approximation that only ever increases towards the true value, without ever overshooting. Here the theorem says that if all the ingredients only ever need to be confirmed from below, the (rescaled) value function inherits that property too.

Proof. We use: if $f(z_1, z_2, \dots)$ is lower semicomputable (Σ^0_1) and monotone increasing in all arguments, and $g_i(x_i)$ are Σ^0_n-computable, then $f(g_1(x_1), g_2(x_2), \dots)$ is Σ^0_n-computable.

Theorem 13.1.3: Lower Semicomputability of V^*_ν, Q^*_ν II

(i) Inserting (6.7.3) for $\pi = \pi^{*,m}_\nu$ into (6.7.5) and multiplying by Γ_t gives

$$\Gamma_t V^{*,m}_\nu(h_{<t}) = \max_{a_t} \sum_{e_t} \nu(e_t|h_{<t}a_t)\big[\gamma_t r_t + \Gamma_{t+1} V^{*,m}_{\nu,\gamma}(h_{1:t})\big],$$

which is Σ^0_n since all components are Σ^0_n, non-negative, and combined only via $\max, \sum, \times, +$ (all monotone increasing), over a finite recursion. Since $V^{*,m}_\nu$ is monotone increasing in m, also $\Gamma_t V^{*,\infty}_\nu(h_{<t}) \in \Sigma^0_n$.

(ii) Multiplying further by $\nu(e_{<t}\|a_{<t})$,

$$\tilde V^*_\nu(h_{<t}) := \nu(e_{<t}\|a_{<t})\Gamma_t V^*_\nu(h_{<t}) = \max_{a_t} \sum_{e_t} \big[\nu(e_{1:t}\|a_{1:t})\gamma_t r_t + \tilde V^*_{\nu,\gamma}(h_{1:t})\big],$$

which is Σ^0_n by the same argument. ■

From Value Functions to Optimal Policies

These results are especially interesting for **Solomonoff's distribution** $\xi_U = M$, which lies in $\mathcal{M}_{sol}$ (Definition 3.7.1), i.e.

$$\xi \in \Sigma_1^0 \subset \Delta_2^0.$$

M is lower semicomputable (Σ_1^0): we can always compute better and better lower bounds on it, we just never know exactly how close we currently are. Being lower semicomputable automatically implies being approximable (Δ_2^0), one level up.

Now that we know the level of computability of the value function, we need to relate it to the computability of an *optimal policy* — the actual decision rule the agent follows.

Theorem 13.1.4: Computability of Optimal Policies

Theorem 13.1.4 (Computability of optimal policies)

For any environment ν, if V_ν^* is Δ_n^0-computable, then there is an optimal (deterministic) policy π_ν^* for the environment ν that is Δ_{n+1}^0-computable.

Proof from [LH18, Thm.17]. To break ties between equally good actions, fix a total order $\succ$ on the action space $\mathcal{A}$ specifying which action wins a tie. Define

$$\pi_\nu(h_{<t}) := a \iff \bigwedge_{a' : a' \succ a} Q_\nu^*(h_{<t}, a) > Q_\nu^*(h_{<t}, a') \ \wedge \bigwedge_{a' : a \succ a'} Q_\nu^*(h_{<t}, a) \geq Q_\nu^*(h_{<t}, a').$$

In plain English: action a is chosen iff it strictly beats every action that is preferred to it in the tie-break order, and at least ties every action it is preferred over. This pins down a unique optimal action at every history.

Then π_ν is ν-optimal. Since V_ν^* is Δ_n^0-computable, $Q_\nu^*(h_{<t}, a) > Q_\nu^*(h_{<t}, a')$ is Σ_n^0 and $Q_\nu^*(h_{<t}, a) \geq Q_\nu^*(h_{<t}, a')$ is Π_n^0. Therefore π_ν, being a conjunction of a Σ_n^0 statement and a Π_n^0 statement, is itself Δ_{n+1}^0. ■

Careful! The Hierarchy Climbs Fast — and ε-Optimality Saves a Level

This shows that if we are not careful, we quickly climb the arithmetic hierarchy: Solomonoff's $M \in \Delta_2^0$, which brings the iterative value (Definition 6.6.1) up to Δ_3^0 due to the $m \to \infty$ limit, resulting in $\pi_M^* \in \Delta_4^0$ for *infinite-horizon* AIXI.

On the other hand, Theorem 13.1.4 only states that *some* optimal policy is Δ_{n+1}^0; other optimal policies might have a strictly lower level. Trying to determine whether two actions have the *exact* same value is a wasteful use of resources — usually it is fine to take a slightly suboptimal action. We therefore consider policies guaranteed to be within $\varepsilon > 0$ (epsilon) of optimal value.

Punchline

ε-optimal policies (Definition 8.1.2) require *one less level* of computability than exactly-optimal policies.

Theorem 13.1.5: Computability of ε-Optimal Policies I

Theorem 13.1.5 (Computability of ε-optimal policies)

For any environment ν, if V_ν^* is Δ_n^0-computable, then for all $\varepsilon > 0$ there is an ε-optimal (deterministic) policy π_ν^ε for the environment ν that is Δ_n^0.

Proof from [LH18, Thm.20]. Let $\varepsilon > 0$. Since V_ν^* is Δ_n^0-computable, construct $Q_\varepsilon := \{(h, a, q) \in \mathcal{H} \times \mathcal{A} \times \mathbb{Q} \mid |q - Q_\nu^*(h, a)| < \varepsilon/2\}$, itself Δ_n^0. Fix a total tie-break order $\succ$ on $\mathcal{A}$. WLOG $\varepsilon = 1/k$; let Q be the $\varepsilon/2$-grid of $[0, 1]$, $Q = \{0, 1/2k, 2/2k, \ldots, 1\}$. Define

$$\begin{aligned}\pi_\nu^\varepsilon(h_{<t}) := a \iff \exists (q_{a'})_{a' \in \mathcal{A}} \in Q^{|\mathcal{A}|}.\ & \bigwedge_{a' \in \mathcal{A}} (h, a', q) \in Q_\varepsilon \\ & \wedge \bigwedge_{a' : a' \succ a} q_a > q_{a'} \ \wedge \bigwedge_{a' : a \succ a'} q_a \geq q_{a'} \\ & \wedge \text{ the tuple is lexicographically minimal in } Q^{|\mathcal{A}|}.\end{aligned}$$

Theorem 13.1.5: Computability of ε-Optimal Policies II

In plain English: round every Q-value to the nearest point on a fine grid (to within $\varepsilon/2$), then pick the action with the highest rounded value, breaking ties lexicographically. Rounding removes the need to know values exactly, which is what saves a level of the hierarchy.

The choice of a is unique; since $\mathcal{A}$ is finite, $Q^{|\mathcal{A}|}$ is finite too, so π_ν^ε is Δ_n^0. ■

Corollary 13.1.6: Putting It Together — Computability of AIXI

Corollary 13.1.6 (Computability of AIXI)

$$\pi^{\varepsilon}_{M} \in \Delta^{0}_{2} \qquad \text{and} \qquad \pi^{*}_{M} \in \Delta^{0}_{3}.$$

In words: there is an ε-optimal policy for AIXI that is *approximable* (Δ^{0}_{2}). There is an optimal AIXI policy that is Δ^{0}_{3}.

This is the headline positive result of the chapter: although AIXI itself is incomputable, we now know precisely how far up the arithmetic hierarchy we must go to recover it — two levels for "almost" AIXI, three levels for exact AIXI.

Theorem 13.1.7: No AIXI Is Computable I

Theorem 13.1.7 (No AIXI is computable)

(Deterministic) AIXI is incomputable, i.e. $\pi^*_\xi \notin \Delta^0_1$.

Proof (by contradiction). Assume π^*_ξ is computable. Since it is deterministic and computable, we can build an adversarial environment μ (mu) that rewards the agent 0 as long as it follows π^*_ξ, and rewards it 1 forever after the *first* time it deviates:

$$\mu(r_t|h_{<t}a_t) := \begin{cases} 1 & \text{if } \forall k \leq t,\ a_k = \pi^*_\xi(h_{<k}) \text{ and } r_k = 0 \\ 1 & \text{if } \forall k \leq t,\ r_k = [\![k \geq i]\!] \text{ where } i := \min\{j \mid a_j \neq \pi^*_\xi(h_{<j})\} \\ 0 & \text{otherwise} \end{cases}$$

$[\![\cdot]\!]$ is the indicator function (1 if the condition holds, else 0). Because μ is built from π^*_ξ itself, an agent that knows and follows π^*_ξ earns reward 0 forever in μ — even though a trivially better policy (deviate once) would earn reward 1 forever after.

Theorem 13.1.7: No AIXI Is Computable II

This gives $V_\mu^{\pi_\xi^*}(h_{<t}) = 0$ for all $h_{<t}$. Since μ satisfies the conditions of Theorem 7.4.10, $V_\xi^*(h_{<t}) \not\to 0$ on histories generated by μ and π_ξ^*. But this contradicts Theorem 7.3.1, which guarantees

$$V_\xi^*(h_{<t}) - V_\mu^{\pi_\xi^*}(h_{<t}) \to 0 \quad \text{for } t \to \infty,\ \mu^{\pi_\xi^*}\text{-almost surely.} \qquad \blacksquare$$

Theorem 13.1.8 and Why Incomputability Is Not the End of the Story

Theorem 13.1.8 (ε-AIXI is not computable)

$\varepsilon\text{-}AIXI \notin \Delta_1^0$.

Proof. See [LH18, Thm.25]. ■

While it is not ideal that the "most intelligent" general agent is incomputable, this is hardly surprising: it follows directly from our choice of what "general" means — the (infinite) class of *all* semicomputable semimeasures. Choosing *smaller* classes $\mathcal{M}$ in $\mathrm{AI}\xi$ is exactly what made MDP-AIXI and MC-AIXI-CTW computable (Chapters 11–12).

The primary point of AIXI was never that we expected to run it on our computer, but that it serves as an **ideal** — the gold standard of the best possible agent we could ever hope to achieve, not the best agent practically achievable. The same holds for ε-optimal policies as $\varepsilon \to 0$.

Computability of Extensions of AIXI

Agents similar to AIXI, such as knowledge-seeking agents (Definition 9.3.2) and BayesExp (Algorithm 9.4), have similar computability levels.

Theorem 13.1.9 (Computability of knowledge-seeking policies)

For entropy-seeking and information-seeking agents, there are approximable (Δ_2^0) ε-optimal policies and Δ_3^0-computable optimal policies.

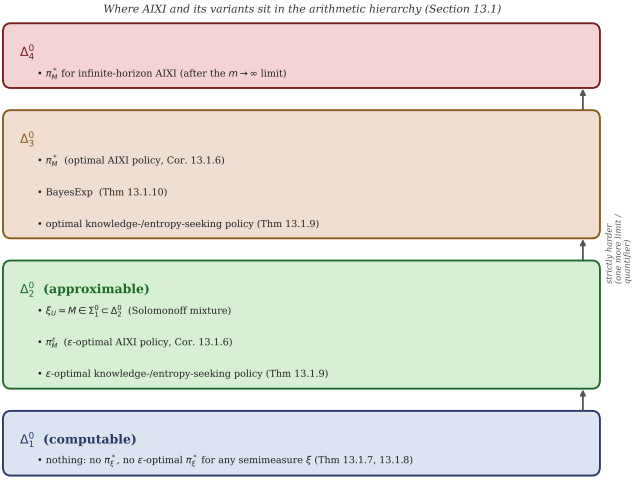
Proof. Follows directly from Theorems 13.1.4 and 13.1.5, letting the environment ν be the knowledge-seeking environment. ■

Theorem 13.1.10 (BayesExp is Δ_3^0)

For any universal mixture ξ, BayesExp is Δ_3^0.

Although Δ_3^0 is far from realistically computable, the fact that ε-optimal policies are approximable (Δ_2^0) shows we have limit-computable algorithms for reasonably-optimal general agents [LH18, Thm.39].

Map: Where AIXI and Its Variants Sit in the Hierarchy



Python: Estimating V_ν^* via Finite-Depth Recursion

We make the proof of Theorem 13.1.1(ii) concrete: a toy environment with a *constant* per-step reward $\bar{r} = 0.7$ and geometric discounting $\gamma_k = \gamma^k$, $\gamma = 0.9$. The recursion telescopes to $V_\nu^*(h_{<t}) = \bar{r}$ exactly, while the finite-depth approximation $V_\nu^{*,m}(h_{<t})$ truncates the sum at depth m. We check numerically that the gap always respects the proven bound Γ_{m+1}/Γ_t.

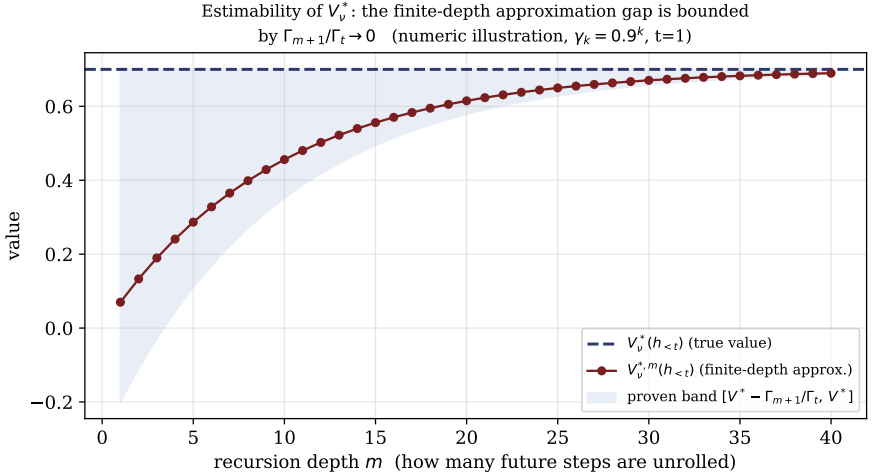
```python
gamma, r_bar, t = 0.9, 0.7, 1
Gamma = lambda k: gamma**k / (1 - gamma)       # discount normalizer
Gt = Gamma(t)

def V_star_m(m):                               # finite-depth value, depth m
    s = sum(gamma**k * r_bar for k in range(t, m + 1))
    return s / Gt

V_true = V_star_m(100000)                       # essentially V*_nu(h<t)
for m in [1, 5, 10, 20, 40]:
    Vm    = V_star_m(m)
    bound = Gamma(m + 1) / Gt                    # proven bound Gamma_{m+1}/Gamma_t
    gap   = V_true - Vm
    print(f"m={m:3d}  V*,m={Vm:.6f}  gap={gap:.6f}  bound={bound:.6f}")
```

Python: Output — the Gap Always Respects the Proven Bound

```
V*_nu(h<1) ~ 0.700000
m=   1   V*,m=0.070000   gap=0.630000   bound=0.900000
m=   5   V*,m=0.286657   gap=0.413343   bound=0.590490
m=  10   V*,m=0.455925   gap=0.244075   bound=0.348678
m=  20   V*,m=0.614896   gap=0.085104   bound=0.121577
m=  40   V*,m=0.689653   gap=0.010347   bound=0.014781
```

The gap is always $\leq$ the proven bound, and both shrink to 0 — exactly the mechanism that makes V_ν^* estimable and hence Δ_n^0 whenever $\nu, \Gamma, \mathcal{R}$ are.

Numeric Illustration of Estimability



Potential Approaches to Approximate AIXI I

Knowing that AIXI is not computable, we want to know: what is the *closest* we can get to AIXI while remaining finitely computable?

Recap of earlier approximations. MDP-AIXI and MC-AIXI-CTW replace ξ_U by a computable ξ_{MDP} or ξ_{CTW}, and replace expectimax by MCTS. Downsides:

(i) The environment class $\mathcal{M}$ is fixed *a priori* and immutable, even if evidence suggests the true environment lies outside $\mathcal{M}$, or if extra compute becomes available to handle a larger class.

(ii) For some problems, solving learning and planning *jointly* is significantly more efficient than approximating ξ and expectimax separately (e.g. certain Bayesian bandit problems have closed-form solutions).

The ideal. We want an *anytime* algorithm that approaches AIXI in the limit of unlimited compute, and is best-in-class at every finite budget. This is where **AIXI**tl [Hut05b] comes in: it finds the program which, when run, leads to an agent closest to AIXI, among programs that halt within time t and have length at most l.

Potential Approaches to Approximate AIXI II

Why not just convert Corollary 13.1.6 into an anytime algorithm? Some versions of AIXI are limit-computable (Corollary 13.1.6), so in principle the longer one spends (per time step) approximating the value function, the more accurate it becomes, converging in the limit to the exact value $Q^*_\xi(h_{<k}, a_k)$ and $\varepsilon_k \to 0$-optimal action. Unfortunately this is totally impractical: this algorithm converges *slower than any computable function*.

Another approach: fix the best policy once and for all. Consider the finite class of all time- and length-bounded policies Π_{tl}, and determine the best-in-class policy *a priori*,

$$\pi^{tl}_\xi := \arg\max_{\pi \in \Pi_{tl}} V^\pi_\xi(\epsilon),$$

then use it for the agent's entire lifetime. Two problems:

(i) We cannot compute π^{tl}_ξ, since $V^\pi_\xi(\varepsilon)$ is incomputable — finding π^{tl}_ξ is itself solving the AGI problem in practice.

Potential Approaches to Approximate AIXI III

(ii) Fixing a *single* time-bounded policy that must a priori handle every eventuality life throws at the agent may be wasteful. Finding more specialized policies the more the agent knows seems more promising (e.g. a chess engine). Note also that time-consistency no longer holds ($\pi^{tl}_{\xi,k} \neq \pi^{tl}_{\xi,1}$, Remark 6.6.2).

AIXItl addresses both problems. There are direct policy-search algorithms in RL that do not maintain a value function [Wil92, KHS01a], but these are the exception. It seems reasonable, when considering Bayes-optimal agents π^*_ξ, to assume the value function needs to be approximated well. So instead we consider an **extended policy** that outputs both an action *and* an approximation of its own current value: $\dot{\pi} : \mathcal{H} \to \mathcal{A} \times [0,1]$. At time step k, write $a^{\dot{\pi}}_k = \dot{\pi}(h_{<k})_a$ for the action taken, and $v^{\dot{\pi}}_k = \dot{\pi}(h_{<k})_v$ for the claimed value estimate.

We write $\ell(\dot{\pi})$ and $t(\dot{\pi})$ for the *length* and *time* of an extended policy: the length of the program p with $U(ph_{<t}) = \dot{\pi}(h_{<t})$ for some universal Turing machine U, and the time it takes $U(ph_{<t})$ to halt. We only consider extended policies whose programs halt in time $\tilde{t}$ and have length at most $\tilde{l}$.

(In this section, k is used as the time index, reserving t to denote a compute-time budget.)

Definition 13.2.1: Valid Approximation

Although an extended policy may take "good" actions, we would like to categorize the class of policies with "valid" approximations of the value function: those that never *overrate* themselves.

Definition 13.2.1 (Valid Approximation)

The logical predicate $\mathrm{VA}(\dot{\pi})$ is true iff $\dot{\pi}$ always satisfies $v^{\dot{\pi}} \leq V_\xi^{\dot{\pi}}$, i.e.

$$\mathrm{VA}(\dot{\pi}) \iff \left[\forall k\, \forall h_{<k} \in \mathcal{H} :\ \dot{\pi}(h_{<k})_v \leq V_\xi^{\dot{\pi}}(h_{<k})\right].$$

In plain English: a valid extended policy may underestimate its own true value $V_\xi^{\dot{\pi}}$, but it is never allowed to overclaim — its self-reported value estimate $v^{\dot{\pi}}$ must always be a lower bound on the value it can actually deliver.

We only consider policies that underrate or exactly rate themselves, for two reasons: (i) we want to select policies with high value, but allowing overconfidence would let us select overrated policies whose actual value is much lower; (ii) combined with a lower semicomputable choice of V_ξ^π, this ensures AIXItl converges (in some sense) to AIXI in the limit $t, l \to \infty$ (Theorem 13.2.4).

Definition 13.2.2: Effective Intelligence Order Relation

Assuming an extended policy $\dot{\pi}$ is a valid approximation, we want the "best" $\dot{\pi}$ computable with time t and length l. This needs an order relation on policies based on their claimed value.

Definition 13.2.2 (Effective intelligence order relation)

Valid $\dot{\pi}$ is called *effectively more or equally intelligent* than valid $\dot{\pi}'$, written $\dot{\pi}' \preceq_c \dot{\pi}$, if

$$\dot{\pi}' \preceq_c \dot{\pi} \iff \big[\forall k \, \forall h_{<k} \in \mathcal{H} : \; \dot{\pi}(h_{<k})_v \leq \dot{\pi}'(h_{<k})_v\big].$$

Careful with the direction: $\dot{\pi}' \preceq_c \dot{\pi}$ means $\dot{\pi}$'s claimed value is at least as large as $\dot{\pi}'$'s at every history — $\dot{\pi}$ claims to know a better lower bound on itself, so it is judged "smarter."

This order relation is a *self-belief* version of the (incomputable) total **Legg–Hutter intelligence measure** $\Upsilon(\pi)$ (Upsilon, Definition 16.7.1): it measures how intelligent the extended policy *believes itself* to be, rather than how intelligent it truly is.

$\preceq_c$ is an **upper semicomputable partial order** on extended policies: it is reflexive, antisymmetric, and transitive, and (as a function) upper semicomputable. Without resource restrictions, it would still have a global maximum $\dot{\pi} \preceq_c \dot{\pi}^*_\xi := (\pi^*_\xi, V^*_\xi)$ for all valid $\dot{\pi}$.

The AIXItl Algorithm — Setup Phase I

AIXItl takes as inputs a max time $\tilde{t}$, a max length $\tilde{l}$, and a max proof length l_P (used for the maximum length of correctness proofs).

Setup (before interaction begins). Check through all $b \in \mathbb{B}^{l_P}$ in some order, testing whether b constitutes a *proof* that some $\dot{\pi}$ of length at most $\tilde{l}$ is a valid extended policy. "Proof" means: b is interpreted as a proof in the same formal axiomatic system in which $\mathrm{VA}(\cdot)$ is defined. Every recorded pair $(b, \dot{\pi})$ contributes $\dot{\pi}$ to the set Π_{VA} of all such proven-valid policies.

This setup takes $O(l_P^2\, 2^{l_P})$ time: checking 2^{l_P} candidate proofs, each in time $O(l_P^2)$. Crucially, this cost is unrelated to $\tilde{t}$ — it only depends on the proof-search budget l_P. It requires significant compute, and some justification for why a voting-style algorithm over machine-checked proofs is an unavoidable ingredient.

The AIXItl Algorithm — Setup Phase II

We then modify every $\dot{\pi} \in \Pi_{\mathrm{VA}}$ so that, during any cycle k, if $\dot{\pi}$ does not halt within time $\tilde{t}$, it is forced to output $(a, 0)$ for an arbitrary a — claiming value 0, which is automatically valid. This guarantees every $\dot{\pi} \in \Pi_{\mathrm{VA}}$ runs in time at most $\tilde{t}$ and remains valid.

Algorithm 13.1: AIXItl

```
Algorithm 13.1  AIXItl [Hut05b]
Require: Max time t~, max program length l~, max proof length l_P
Input:   Percept stream e1, e2, e3, ...
Output:  (t~, l~, l_P)-optimal action stream a1, a2, a3, ...

 1: Interpret all b in B^{l_P} as proofs of VA(pi-dot) for some pi-dot
 2: Let Pi_VA be the set of those pi-dot's such that
      length(pi-dot) <= l~  and  VA(pi-dot) has been proven
 3: Modify each pi-dot so that, in cycle k, if it does not output
      (a_k, v_k^{pi-dot}) within time t~, force it to output (a, 0)
 4: Update Pi_VA to be the set of these new pi-dot's
 5: Set k := 1
 6: for k = 1, 2, 3, ... do
 7:     for pi-dot in Pi_VA do
            run pi-dot on h_{<k}; receive pi-dot(h_{<k}) = (a_k^{pi-dot}, v_k^{pi-dot})
 8:     Choose pi-dot* = argmax_{pi-dot in Pi_VA} v_k^{pi-dot}   (highest claimed value)
 9:     Write a_k := a_k^{pi-dot*}  on the output tape
10:     Receive input e_k from the environment
```

Every cycle, AIXItl runs every proven-valid extended policy on the current history, and simply acts according to whichever one currently claims the highest value. Because every $\dot{\pi}$ in Π_{VA} is provably valid (never overrates itself), trusting the loudest claim is provably safe.

AIXItl: Why Trusting the Highest Claim Works I

Since every $\dot{\pi} \in \Pi_{\mathrm{VA}}$ has a proof of $\mathrm{VA}(\dot{\pi})$, we know that for all k,

$$v_k^{\dot{\pi}} \leq V_\xi^{\dot{\pi}}(h_{<k}).$$

This means picking the $\dot{\pi}$ with the largest $v_k^{\dot{\pi}}$ leads to the most intelligent agent according to the effective intelligence order relation $\preceq_c$. Additionally, since at each interaction cycle we loop through Π_{VA}, whose *size* is independent of $\tilde{t}$, each cycle takes time $O(\tilde{t})$ per policy evaluated — i.e. the same $\tilde{t}$ budget, repeated $|\Pi_{\mathrm{VA}}|$ times.

Theorem 13.2.3 (Optimality of AIXItl [Hut05b])

Let $\dot{\pi}$ be any extended policy of length $\ell(\dot{\pi}) \leq \tilde{l}$ and computation per cycle $t(\dot{\pi}) \leq \tilde{t}$, for which there exists a proof of $\mathrm{VA}(\dot{\pi})$ of length $\leq l_P$. Algorithm 13.1, which depends on $\tilde{l}, \tilde{t}, l_P$ but *not* on $\dot{\pi}$, is effectively more or equally intelligent according to $\preceq_c$ than any such $\dot{\pi}$.

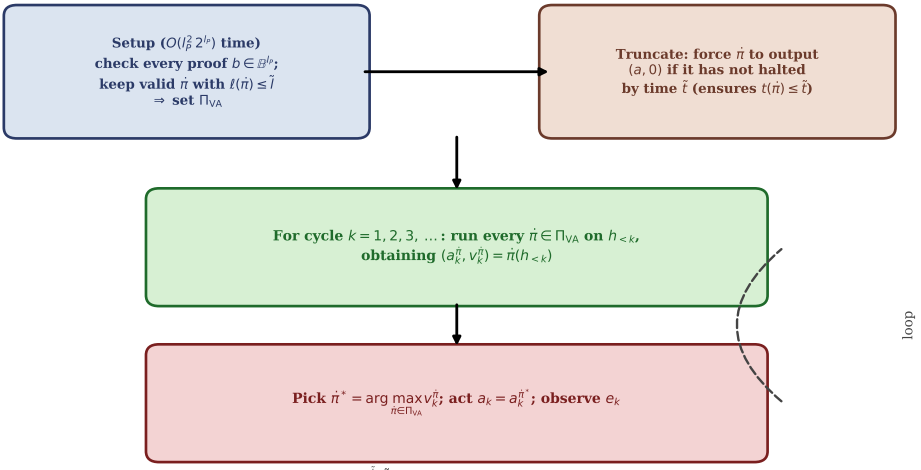
The length of the optimal policy found, $\dot{\pi}^*$, is $\ell(\dot{\pi}^*) = O(\log(\tilde{l} \cdot \tilde{t} \cdot l_P))$; the setup time is $t_{\mathrm{setup}}(\dot{\pi}^*) = O(l_P^2\, 2^{l_P})$; and the computation time per cycle is $t_{\mathrm{cycle}}(\dot{\pi}^*) = O(2^{\tilde{l}} \cdot \tilde{t})$.

AIXItl: Why Trusting the Highest Claim Works II

Proof. A direct consequence of the construction: at each time step, if $\dot{\pi}$ is the most effectively intelligent policy in Π_{VA}, AIXItl acts according to it; if not, AIXItl acts as the more effectively intelligent policy would. Either way, AIXItl is effectively more or equally intelligent according to $\preceq_c$. ■

Note: potentially following a different policy at each time step might seem counterintuitive, but it does not matter for AIXItl, whose only objective is to maximize its effective intelligence at each individual step, using whatever is available in Π_{VA} at that moment.

The AIXItl Interaction Loop, Visualized



Per-cycle cost $O(t_{\mathrm{cycle}}) = O(2^{\tilde{l}} \cdot \tilde{t})$ — independent of the number of elapsed cycles k

Just How Impractical? And Does AIXItl Approach AIXI?

Although the setup time and time per cycle are rather large, if we fix l_P and $\tilde{l}$, the setup time is *constant* in $\tilde{t}$ and the time per cycle is $O(\tilde{t})$ — although these hidden constants are extremely (astronomically) large in practice.

Does AIXItl approach AIXI as the budgets grow? A key requirement is that $V^*_\xi(h_{<k})$ be lower semicomputable. Theorem 13.1.3 gives two routes (for $n = 1$): (i) we would need $\xi(e_k|h_{<k}a_k)$ itself lower semicomputable — but $\xi \in \mathcal{M}_{sol}$ does *not* satisfy this (a construction similar to Section 10.5 may still work); (ii) we only need the *joint* $\xi(e_{<t}\|a_{<t})$ lower semicomputable, which *does* hold for $\xi = \xi_U$. This gives us $\tilde{V}^*_{\xi_U}(h_{<t}) := \xi_U(e_{<t}\|a_{<t})\Gamma_t V^*_{\xi_U}(h_{<t})$, lower semicomputable, which can replace V^*_ξ in Definition 13.2.1 without affecting the effective intelligence order relation (both $\dot{\pi}$'s are scaled by the same factor, independent of a_t).

Theorem 13.2.4 (AIXItl approaches AIXI)

If the scaled optimal value function $\tilde{V}^*_\xi(h_{<k})$ is used and is lower semicomputable, then the behavior of AIXI$\tilde{t}\tilde{l}$ approaches the behavior of AIXI in the limit $\tilde{t}, \tilde{l}, l_P \to \infty$, in some sense.

Proof Sketch of Theorem 13.2.4, and Conclusion I

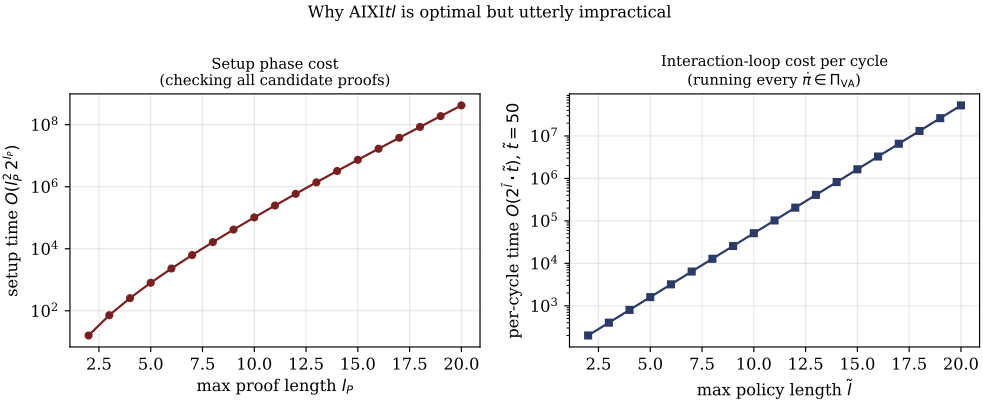
Proof sketch. Lower semicomputability of $V^*_\xi(h_{<k})$ ensures the existence of a sequence of extended programs $\dot\pi_1, \dot\pi_2, \dot\pi_3, \ldots$ for which $\mathrm{VA}(\dot\pi_i)$ can be proven and $\lim_{i\to\infty} v_k^{\dot\pi_i} = V^*_\xi(h_{<k})$ for all k and histories $h_{<k}$. One such sequence: terminate some (non-halting) lower-semicomputing scheme for $Q^*_\xi(h_{<k}, a_k)$ after i time steps, and use the obtained approximation as $v_k^{\dot\pi_i}$ for the action $a_k^{\dot\pi_i}$ maximizing the approximate Q-value. The convergence $v_k^{\dot\pi_i} \to V^*_\xi(h_{<k})$ ensures that the universally optimal value can be approximated by *some* provably valid $\dot\pi$ arbitrarily well, given enough time and length. The approximation is not uniform in k, but this does not matter, as the selected $\dot\pi$ is allowed to change from interaction cycle to cycle. ■

Proof Sketch of Theorem 13.2.4, and Conclusion II

Conclusion

Similar to other reinforcement learning algorithms, AIXItl evaluates policies, but in an unusual, **provably pessimistic** way: it insists on a proof of correctness (valid approximation) before comparing a policy to every other valid, non-overconfident policy. Crucially, AIXItl does not require $\dot{\pi}$ to first estimate a model ξ_U and then compute $V^*_{\xi_U}$ separately — it can estimate the latter directly, model-free, or in whatever way is most efficient. While dovetailing through all programs to find the best is a well-known idea, making it work for AIXI was non-trivial, since $V^*_{\xi_U}$ itself is incomputable; lower semicomputability of the scaled $\tilde{V}^*_{\xi_U}$ was crucial. Roughly: if some unknown policy π of size $\tilde{l}$ and per-cycle time $\tilde{t}$ has certain capabilities (chess master, AGI, ASI), then the known, computable AIXI$\tilde{t}\tilde{l}$ has the same capabilities within the same time frame, up to an (unfortunately very large) constant factor.

Why AIXItl Is Optimal but Utterly Impractical



Python: A Toy Universal Search over Π_{VA}

A miniature illustration of the mechanism at the heart of AIXItl: we enumerate every candidate "extended policy" as a bit-string of length $\leq \tilde{l}$ (a stand-in for the finitely many proven-valid programs in Π_{VA}), let each one output a deterministic "self-claimed value" $v^{\tilde{\pi}}$, and pick the $\arg\max$ — exactly line 8 of Algorithm 13.1.

```
import itertools

def claimed_value(bits):                          # toy deterministic self-rating
    h = sum((i + 1) * int(b) for i, b in enumerate(bits))
    return (h % 97) / 97                           # a pseudo-value in [0,1)

l_tilde = 4                                        # max program length
candidates = [''.join(bits)
              for length in range(l_tilde + 1)
              for bits in itertools.product('01', repeat=length)]
print(f"|Pi_VA| = {len(candidates)} candidates of length <= {l_tilde}")

scored = sorted(((pi, claimed_value(pi)) for pi in candidates),
                key=lambda x: -x[1])
for pi, v in scored[:5]:
    print(f"  pi_dot={pi or '(empty)':>6s}   v={v:.4f}")

best_pi, best_v = scored[0]
print(f"AIXItl acts as pi_dot* = {best_pi!r}, claimed value {best_v:.4f}")
```

Python: Output — Picking the Highest Claimed Value

```
|Pi_VA| = 31 candidates of length <= 4
  pi_dot=  1111   v=0.1031
  pi_dot=  0111   v=0.0928
  pi_dot=  1011   v=0.0825
  pi_dot=  0011   v=0.0722
  pi_dot=  1101   v=0.0722
AIXItl acts as pi_dot* = '1111', claimed value 0.1031
```

With $\tilde{l} = 4$ there are already $31 = 2^5 - 1$ candidates; the `search_cost` figure shows this count exploding as $O(2^{\tilde{l}})$ — exactly why the theorem's constants are astronomically large.

13.3 Exercises I

1. [C30] Prove (or disprove): for any environment ν, if V_ν^* is Δ_n^0-computable, then there exists an optimal (deterministic) policy π_ν^* for the environment ν that is *not* Δ_n^0-computable.
2. [C32] Prove (or disprove): for any environment ν, if V_ν^* is Δ_n^0-computable, then for $\varepsilon > 0$ there is *no* ε-optimal (deterministic) policy π_ν^ε for the environment ν that is Δ_{n-1}^0.
3. [C22] Prove Corollary 13.1.6.
4. [C26] Prove Theorem 13.1.8, that ε-AIXI is not computable.
5. [C40] Various results of this chapter assumed deterministic policies. Prove (or disprove) that they also hold for stochastic policies.
6. [C26] Prove that BayesExp is Δ_3^0.
7. [C40] Derive the computability bounds for Thompson sampling.

The bracketed numbers such as [C30] are the book's difficulty ratings for each exercise (roughly, "C" for a problem requiring creative/compositional reasoning, with the number indicating relative difficulty/length).

13.4 History and References I

Computability in RL. This chapter is based on material from [LH18] and [Lei16b, Chp. 6]. Many results around the computability of AIXI and other UAI agents were first derived in [LH15c, LH15d] and later extended in [Lei16b, LH18]. While this is the first work showing computability results in the UAI framework, traditional RL has many computability and complexity-theory results of its own. On the complexity-theory side, planning is P-complete in MDPs with finite and infinite horizons [PT87]. [MGLA00] proved (among other results) that deciding whether a sufficiently good policy exists in MDPs and POMDPs is PSPACE-complete. [LGM01] showed that (unless the polynomial hierarchy collapses, e.g. P=NP) optimal policies for MDPs and POMDPs are not ε-approximable. [SLR07] showed that deciding whether there exists a policy with value exceeding a given value in epistemic POMDPs is PSPACE-complete, as well as the existence of an optimal policy in an epistemic POMDP being NP. On the computability side, [MHC99, MHC03] showed that planning with infinite horizon in general POMDPs is even formally undecidable.

On the topic of Solomonoff induction, [Net22] argued that the problems of the relative choice of machine and incomputability in Solomonoff prediction are both caused by the fact that computable approximations of Solomonoff prediction do not always converge.

13.4 History and References II

Regarding halting and termination of programs, [Ica17] showed that when using probabilistic computation in cognitive science, it is not always desirable to have models which always terminate, and argues for some benefits of non-terminating models.

13.4 History and References III

AIXItl. AIXItl (Section 13.2) was first introduced in [Hut00] and described again in [Hut03d, Hut05b, Hut07f, Hut12b, Lei16b]. [Pan08] developed a more practical approximation based on Monte Carlo Tree Search sampling programs. Further replacing the Solomonoff prior by CTW led to MC-AIXI-CTW (Chapter 12). [Kat18] introduced a more functional approximation based on typed lambda calculus and argued it to be even more powerful than AIXItl.

13.4 History and References (cont'd): Optimal Solvers and Universal Search I

One key component of AIXItl is that of **universal (Levin) search**. Introduced in [Lev73c], universal search is a method designed to efficiently find a program that can solve a problem, provided the solution can be efficiently verified.

There have been many extensions of Universal Search:

- An adaptive extension of Levin Search was introduced in [WS96], used to solve POMDPs.
- [Hut01b, Hut02b] extends Levin search to a provably fastest algorithm for all well-defined problems, even if the solution cannot be verified efficiently.
- [Sch04] develops a more practical Adaptive version of Levin Search (ALS), called the **Optimal Ordered Problem Solver** (OOPS), which can take advantage of previous solutions.
- **Levin Tree Search** (LTS) [OLLW18] is a policy-guided search algorithm with an adaptive policy that comes with theoretical guarantees. The policy may be represented using neural networks (LTS+NN) [OL21], or parameterized in a convex way using context models (LTS-CM) [OHL23], with theoretical guarantees and award-winning practical performance on Sokoban, The Witness, and the 24-Sliding Tile puzzle.

13.4 History and References (cont'd): Optimal Solvers and Universal Search II

Also related to Universal Search is the **Gödel Machine** [Sch07], a formal self-improving optimal problem solver. [Sch05] argued how the Gödel Machine could be used for a formal definition of consciousness. Several potential implementations of the Gödel Machine are discussed in [SS11].